

# **Part 6 — Advanced and production: Bayes, Deep Learning, persistence and reporting**

**Complete tutorial for sciModelFlowR (version 1.0.2)**

Walter Esfrain Pereira

2026-09-19

# Table of contents

|          |                                                                         |           |
|----------|-------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Bayesian models: from prior to posterior, with typed uncertainty</b> | <b>3</b>  |
| 1.1      | GP and BART: functions the line cannot see . . . . .                    | 5         |
| <b>2</b> | <b>Deep Learning with a reproducibility contract</b>                    | <b>7</b>  |
| <b>3</b> | <b>Production: track, persist, serve</b>                                | <b>10</b> |
| <b>4</b> | <b>Reporting and governance</b>                                         | <b>13</b> |
| <b>5</b> | <b>Synthesis: the complete flow in one table</b>                        | <b>15</b> |
| <b>6</b> | <b>Appendix A — API coverage, computed</b>                              | <b>16</b> |
| <b>7</b> | <b>Appendix B — Minimal glossary</b>                                    | <b>22</b> |
| <b>8</b> | <b>Appendix C — Execution environment</b>                               | <b>23</b> |
| <b>9</b> | <b>Part 6 exercises</b>                                                 | <b>24</b> |

# 1 Bayesian models: from prior to posterior, with typed uncertainty

In the small trial (`bayes_small`, `n = 30`), the conjugate normal model allows comparing posterior and truth. The prior is declared, not hidden.

```
bs <- ler("bayes_small")
bsp <- smf_bayesian_spec(
  priors = list(beta_mean = c(5), beta_sd = c(2), sigma_shape = 2, sigma_rate = 1),
  draws = 3000L, seed = 260917L
)
bf <- smf_bayes_fit(yield ~ 1, bs, bayesian = bsp, backend = "conjugate_gaussian")
post <- smf_bayes_summary(bf)
tab_smf(post, caption = "Posterior of the conjugate normal model: mean, SD and quantiles (truth: mean 5.2; SD 1.1).")
```

Table 1.1: Posterior of the conjugate normal model: mean, SD and quantiles (truth: mean 5.2; SD 1.1).

|             | variable    | mean  | sd    | q0.025 | q0.500 | q0.975 |
|-------------|-------------|-------|-------|--------|--------|--------|
| (Intercept) | (Intercept) | 5.287 | 0.182 | 4.928  | 5.288  | 5.651  |
| sigma       | sigma       | 0.978 | 0.122 | 0.774  | 0.963  | 1.256  |

```
mu_grid <- seq(3, 7, length.out = 400)
post_df <- data.frame(
  mu = rep(mu_grid, 2),
  density = c(stats::dnorm(mu_grid, 5, 2),
    stats::dnorm(mu_grid, post$mean[post$variable == "(Intercept)"],
      post$sd[post$variable == "(Intercept)"])),
  distribution = rep(c("prior", "posterior"), each = length(mu_grid))
)
ggplot(post_df, aes(mu, density, colour = distribution)) +
  geom_line(linewidth = 1.1) +
  geom_vline(xintercept = verdade$bayes_small$mu, linetype = "dashed",
    colour = cores_smf[["alerta"]]) +
  scale_colour_manual(values = c(prior = cores_smf[["neutro"]],
    posterior = cores_smf[["primaria"]])) +
  labs(x = "mean yield", y = "density", colour = NULL,
    title = "Prior to posterior: conjugate Gaussian",
    subtitle = paste0("Planted truth: mu = ", fmt(verdade$bayes_small$mu, 1))) +
```

```
theme(legend.position = "top")
```

## Prior to posterior: conjugate Gaussian

Planted truth:  $\mu = 5.2$

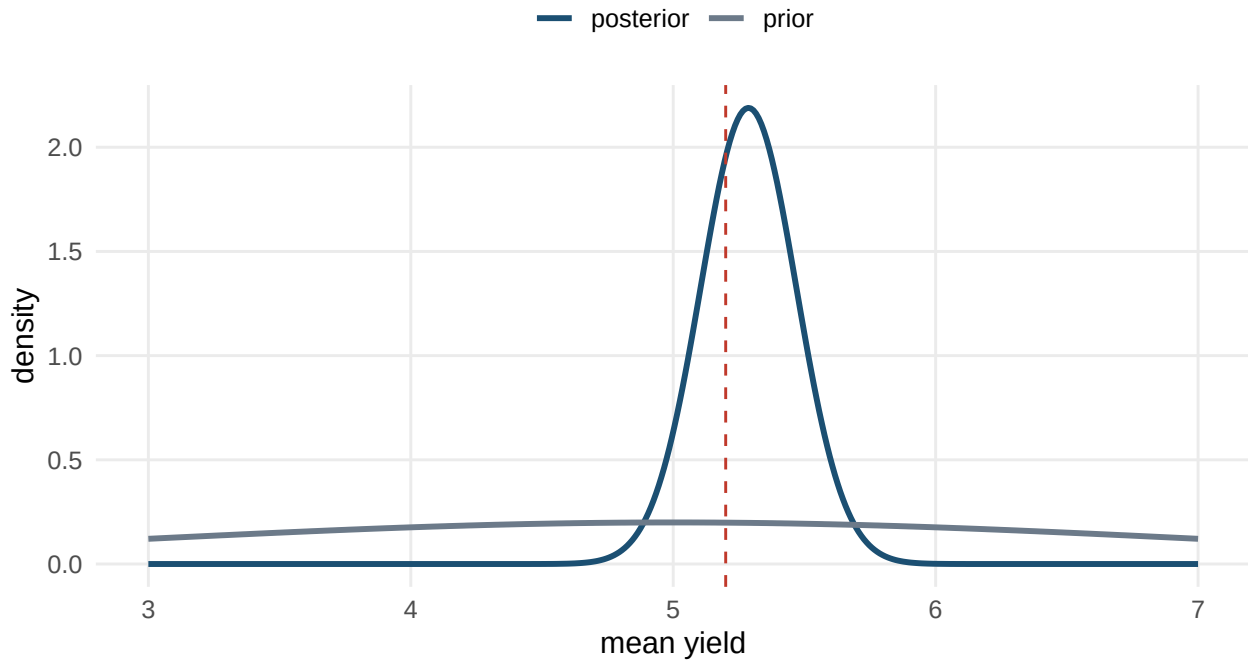


Figure 1.1: Prior (grey) and posterior (blue) for the mean: with  $n = 30$  the likelihood dominates the prior, but the declared prior remains visible.

```
pr_bayes <- smf_bayes_predict(bf, bs[1:5, , drop = FALSE], ndraws = 500L)
cat("representation:", pr_bayes@representation,
    "| interval type:", pr_bayes@uncertainty@interval_type, "\n")
```

representation: samples | interval type: posterior\_predictive

```
ci_bayes <- smf_dist_interval(pr_bayes, level = 0.90)
tab_smf(data.frame(sample = bs$sample_id[1:5], mean = smf_dist_mean(pr_bayes),
                    lower = ci_bayes[, 1], upper = ci_bayes[, 2]),
        caption = "Posterior predictive distribution: mean and 90% interval for five new samples")
```

Table 1.2: Posterior predictive distribution: mean and 90% interval for five new samples.

| sample | mean  | lower | upper |
|--------|-------|-------|-------|
| B01    | 5.188 | 3.604 | 6.704 |
| B02    | 5.266 | 3.601 | 6.817 |

| sample | mean  | lower | upper |
|--------|-------|-------|-------|
| B03    | 5.281 | 3.552 | 6.924 |
| B04    | 5.294 | 3.651 | 7.011 |
| B05    | 5.275 | 3.507 | 6.916 |

```
diag_bayes <- smf_bayes_diagnose(bf)
cat("diagnostic BayesFitResult:", class(diag_bayes)[1], "\n")
```

```
diagnostic BayesFitResult: sciModelFlowR::BayesianDiagnosticResult
```

```
print(diag_bayes)
```

```
<sciModelFlowR BayesianDiagnosticResult>
warnings: 0
```

### **i** Interpretation

The posterior mean is 5.29 (truth 5.2) and the posterior SD is 0.98 (truth 1.1). The predictive interval is **posterior predictive**: it answers “where will a new observation fall”, not “where is the parameter”. Confusing the two is the most common error with Bayesian output — and the package carries the type in the object itself (`interval_type = "posterior_predictive"`).

## 1.1 GP and BART: functions the line cannot see

The Gaussian process smooths the organic-matter map in space; BART captures the nonlinear surface of the `dl_tiny` dataset.

```
soil <- ler("soil_spatial")
set.seed(20260919); gp_order <- sample.int(nrow(soil), 70)
gx <- as.matrix(soil[gp_order[1:60], c("x", "y")])
gy <- soil$organic_matter[gp_order[1:60]]
gp <- smf_gp_fit(gx, gy, optimize = TRUE)
pr_gp <- smf_gp_predict(gp, as.matrix(soil[gp_order[61:70], c("x", "y")]))
rmse_gp <- sqrt(mean((soil$organic_matter[gp_order[61:70]] - smf_dist_mean(pr_gp))^2))
ud_gp <- smf_uncertainty_from_distribution(pr_gp)
tab_smf(data.frame(
  measure = c("GP RMSE (10 out-of-sample points)", "decomposition identified",
    "decomposition method"),
  value = c(fmt(rmse_gp, 3), as.character(ud_gp@identified), ud_gp@method)
), caption = "Spatial GP: out-of-sample error and identified uncertainty decomposition.")
```

Table 1.3: Spatial GP: out-of-sample error and identified uncertainty decomposition.

| measure                           | value       |
|-----------------------------------|-------------|
| GP RMSE (10 out-of-sample points) | 0.837       |
| decomposition identified          | TRUE        |
| decomposition method              | bayesian_gp |

```

dl <- ler("dl_tiny")
x_dl <- as.matrix(dl[, c("x1", "x2")]); y_dl <- dl$y
set.seed(20260919); i_tr <- sample.int(nrow(dl), 240)
bart <- smf_bart_fit(x_dl[i_tr, , drop = FALSE], y_dl[i_tr], task = "regression",
                    ndpost = 300L, nskip = 150L, ntree = 80L, seed = 1L)
pr_bart <- smf_bart_predict(bart, x_dl[-i_tr, , drop = FALSE])
rmse_bart <- sqrt(mean((y_dl[-i_tr] - smf_dist_mean(pr_bart))^2))
ci_bart <- smf_dist_interval(pr_bart, level = 0.90)
cov_bart <- mean(y_dl[-i_tr] >= ci_bart[, 1] & y_dl[-i_tr] <= ci_bart[, 2])
tab_smf(data.frame(
  model = c("BART", "straight line (lm)"),
  rmse = c(rmse_bart, sqrt(mean(lm(y ~ x1 + x2, data = dl[i_tr, ])$residuals^2)))
), caption = "Nonlinear surface: BART learns what the line cannot see (true noise: 0.15).")

```

Table 1.4: Nonlinear surface: BART learns what the line cannot see (true noise: 0.15).

| model              | rmse  |
|--------------------|-------|
| BART               | 0.202 |
| straight line (lm) | 0.742 |

### **i** Interpretation

The GP delivers, besides the prediction, an **identified decomposition** of aleatoric versus epistemic uncertainty (bayesian\_gp), because the model separates noise from function. BART reduces RMSE from 0.742 to 0.202 and covers 73% of observations in the 90% interval — close to nominal, without a formal guarantee. They are different models for different questions: smoothing a spatial field is not the same as learning a response surface.

## 2 Deep Learning with a reproducibility contract

The small MLP (2 -> 32 -> 16 -> 1) learns the planted surface. The training history is a first-class object, not a throwaway plot.

```
architecture <- smf_mlp(input_dim = 2, output_dim = 1, hidden = c(32L, 16L),
  activation = "relu", dropout = 0, task = "regression")
fit_dl <- smf_dl_train(x_dl[i_tr, , drop = FALSE], y_dl[i_tr],
  architecture = architecture, epochs = 150L, batch_size = 32L,
  seed = 20260919L)
history <- fit_dl@history
p_dl <- smf_dl_predict(fit_dl, x_dl[-i_tr, , drop = FALSE])
rmse_dl <- sqrt(mean((y_dl[-i_tr] - p_dl)^2))
tab_smf(data.frame(
  information = c("epochs executed", "best epoch", "best metric", "test RMSE"),
  value = c(history@stopped_epoch, history@best_epoch, history@best_metric, rmse_dl)
), caption = "MLP training record: what reproducibility requires.")
```

Table 2.1: MLP training record: what reproducibility requires.

| information     | value   |
|-----------------|---------|
| epochs executed | 150.000 |
| best epoch      | 150.000 |
| best metric     | 0.092   |
| test RMSE       | 0.327   |

```
hist_df <- data.frame(
  epoch = rep(history@epoch, 2),
  loss = c(history@training, history@validation),
  curve = rep(c("training", "validation"), each = length(history@epoch))
)
ggplot(hist_df, aes(epoch, loss, colour = curve)) +
  geom_line(linewidth = .9) +
  scale_colour_manual(values = c(training = cores_smf[["neutro"]],
    validation = cores_smf[["primaria"]])) +
  labs(x = "epoch", y = "loss", colour = NULL,
    title = "Training history: loss curves") +
  theme(legend.position = "top")
```

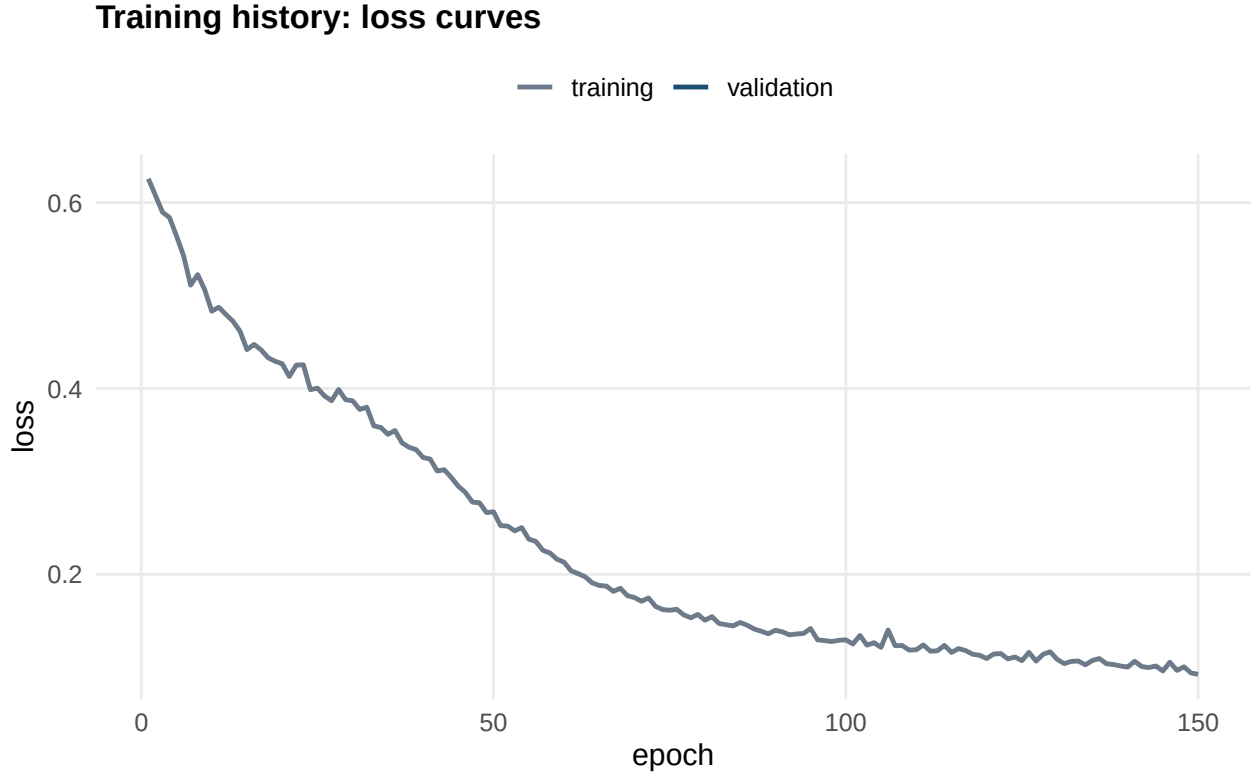


Figure 2.1: Training history: training and validation loss per epoch. Validation is development data — it is not the final evaluation.

```
dist_dl <- smf_dl_predict_distribution(fit_dl, x_dl[-i_tr, , drop = FALSE],
                                     method = "mc_dropout", draws = 50L)
ci_dl <- smf_dl_interval(dist_dl, level = 0.90)
cov_dl <- mean(y_dl[-i_tr] >= ci_dl[, 1] & y_dl[-i_tr] <= ci_dl[, 2])
refusal_dl <- tryCatch(
  smf_dl_uncertainty_decompose(dist_dl, level = 0.90),
  error = function(e) conditionMessage(e))
tab_smf(data.frame(
  measure = c("representation", "90% interval coverage", "decomposition"),
  value = c(dist_dl@representation, fmt(100 * cov_dl, 1), refusal_dl)
), caption = "MLP uncertainty: MC dropout samples and the honest refusal of the unidentified")
```

Table 2.2: MLP uncertainty: MC dropout samples and the honest refusal of the unidentified decomposition.

| measure               | value                                                                                                                        |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------|
| representation        | samples                                                                                                                      |
| 90% interval coverage | 91.7                                                                                                                         |
| decomposition         | Sample variability alone does not identify aleatoric versus epistemic uncertainty; supply a model-based aleatoric component. |

```
grad <- smf_dl_gradient_x_input(fit_dl, x_dl[-i_tr, , drop = FALSE][1:3, , drop = FALSE],
                               target_index = 1L)
tab_smf(data.frame(case = 1:3, x1 = x_dl[-i_tr, ][1:3, 1], x2 = x_dl[-i_tr, ][1:3, 2],
                   d_y_d_x1 = grad@values[, 1], d_y_d_x2 = grad@values[, 2]),
         caption = "Gradient explanation: sensitivity of the prediction to each input (local attribution, not causal).")
```

Table 2.3: Gradient explanation: sensitivity of the prediction to each input (local attribution, not causal).

| case | x1    | x2     | d_y_d_x1 | d_y_d_x2 |
|------|-------|--------|----------|----------|
| 1    | 0.100 | -0.144 | 0.495    | 0.111    |
| 2    | 0.435 | 0.275  | -2.010   | 0.183    |
| 3    | 0.108 | 0.385  | 0.574    | 0.168    |

#### Trap

The package **refused** to decompose MC dropout uncertainty (Sample variability alone does not identify aleatoric versus epistemic uncertainty; supply a model-based aleatoric component.): sample variability alone does not separate aleatoric from epistemic. The honest output is the refusal, not an invented number. For an identified decomposition, use a model that separates the sources (such as the GP above).

### 3 Production: track, persist, serve

A scientific result must survive the session. The production flow has four steps: **record** (tracker), **freeze** (bundle), **serve** (batch prediction with resume) and **report** (manifest and report).

```
field <- ler("field_yield")
v <- verdade$field_yield
no_out <- field[!field$plot_id %in% v$outlier_ids, ]
spec_p6 <- smf_experiment_spec(
  task = smf_task_spec("regression", "yield"),
  data = smf_data_spec("yield", c("nitrogen", "potassium", "water", "organic_matter"),
    id_column = "plot_id"),
  design = smf_design_spec(id_column = "plot_id"),
  preprocessing = smf_preprocess_spec("median"),
  resampling = smf_resampling_spec("holdout", train_prop = 0.7, seed = 21L),
  model = smf_model_spec("linear_regression", "stats"),
  metrics = list(smf_metric_spec("rmse")),
  reproducibility = smf_reproducibility_spec(21L)
)
fit_p6 <- smf_fit_experiment(spec_p6, no_out)

tr_dir <- file.path(tempdir(), "tracker-p6")
unlink(tr_dir, recursive = TRUE)
tr <- smf_tracker_local(tr_dir, experiment = "tutorial", tags = list(part = "6"))
run <- smf_start_run(tr, run_name = "final-model")
run <- smf_log_params(run, list(model = "linear_regression", n = nrow(no_out)))
run <- smf_log_metric(run, "rmse", fit_p6@metrics$value[fit_p6@metrics$metric == "rmse"])
run <- smf_log_result(run, fit_p6)
runs <- smf_list_runs(tr)
tab_smf(utils::head(runs[, c("run_id", "status", "started_at")], 5),
  caption = "Runs recorded in the local tracker (directory path omitted).")
```

Table 3.1: Runs recorded in the local tracker (directory path omitted).

| run_id                       | status  | started_at              |
|------------------------------|---------|-------------------------|
| smf-20260919T035340-42cecaf2 | running | 2026-09-19 03:53:40 UTC |

```
cat("tracker directory:", gsub("\\\\", "/", tr_dir), "\n")
```

tracker directory: C:/Users/wep69/AppData/Local/Temp/Rtmp06CEbu/tracker-p6

```

p_bundle <- file.path(tempdir(), "bundle-p6"); unlink(p_bundle, recursive = TRUE)
smf_save_bundle(fit_p6, p_bundle, mode = "portable")
val_bundle <- smf_validate_bundle(p_bundle)
bundle <- smf_load_bundle(p_bundle)
equal <- isTRUE(all.equal(as.numeric(smf_predict_bundle(bundle, no_out[1:7, ])),
                          as.numeric(smf_predict(fit_p6, no_out[1:7, ]))))
tab_smf(data.frame(
  check = c("valid bundle", "unsafe components", "prediction identical to the live path"),
  result = c(val_bundle@valid, length(val_bundle@unsafe_components), equal)
), caption = "Portable bundle: save, validate, load and predict with numerical equality.")

```

Table 3.2: Portable bundle: save, validate, load and predict with numerical equality.

| check                                 | result |
|---------------------------------------|--------|
| valid bundle                          | 1      |
| unsafe components                     | 0      |
| prediction identical to the live path | 1      |

```

ck_dir <- file.path(tempdir(), "batch-p6"); unlink(ck_dir, recursive = TRUE)
batch1 <- smf_batch_predict(fit_p6, no_out[1:20, ], batch_size = 7L, workers = 2L,
                           checkpoint_dir = ck_dir, resume = FALSE)
batch2 <- smf_batch_predict(fit_p6, no_out[1:20, ], batch_size = 7L, workers = 2L,
                           checkpoint_dir = ck_dir, resume = TRUE)
tab_smf(data.frame(
  measure = c("batch rows", "prediction column", "resume returns the same order",
              "second call marked as resumed"),
  value = c(batch1@n_rows, names(batch1@prediction)[1],
            identical(batch1@prediction$.prediction, batch2@prediction$.prediction),
            batch2@resumed)
), caption = "Batch prediction with checkpoint: resume reuses the work without redoing anything")

```

Table 3.3: Batch prediction with checkpoint: resume reuses the work without redoing anything.

| measure                       | value       |
|-------------------------------|-------------|
| batch rows                    | 20          |
| prediction column             | .prediction |
| resume returns the same order | TRUE        |
| second call marked as resumed | TRUE        |

```

dep <- smf_deployment_spec(adapter = "local", strict_schema = TRUE, batch_size = 128L)
scale <- smf_scalability_spec(chunk_size = 64L, workers = 2L, execution = "sequential",
                             checkpoint_every = 2L)
tab_smf(data.frame(
  contract = c("deployment: adapter", "deployment: strict schema",

```

```

    "scaling: chunk", "scaling: checkpoint every"),
  value = c(dep@adapter, dep@strict_schema, scale@chunk_size, scale@checkpoint_every)
), caption = "Deployment and scaling contracts declared as objects (not loose parameters).")

```

Table 3.4: Deployment and scaling contracts declared as objects (not loose parameters).

| contract                  | value |
|---------------------------|-------|
| deployment: adapter       | local |
| deployment: strict schema | TRUE  |
| scaling: chunk            | 64    |
| scaling: checkpoint every | 2     |

## 4 Reporting and governance

The report is generated from the result, with a completeness checklist and a provenance manifest. Governance closes the cycle: deprecation policy and API catalogue.

```
report <- smf_report(fit_p6, format = "markdown", title = "Complete tutorial - field trial")
report_file <- tempfile(fileext = ".md")
smf_report(fit_p6, format = "markdown", file = report_file)
```

```
<sciModelFlowR ReportingResult>
format: markdown
path: C:\Users\wep69\AppData\Local\Temp\Rtmp06CEbu\file7f1050e66553.md
```

```
checklist <- smf_reporting_checklist(fit_p6)
tab_smf(utils::head(checklist, 10), caption = "Reporting checklist: what was verified and what remains to be declared")
```

Table 4.1: Reporting checklist: what was verified and what remains to be declared.

| item                                            | status      | evidence                          |
|-------------------------------------------------|-------------|-----------------------------------|
| scientific question/task                        | required    | regression                        |
| experimental or sampling design                 | required    | DesignSpec                        |
| data audit and leakage safeguards               | required    | DataAuditResult                   |
| resampling and final assessment separation      | required    | SplitRecord/ResampleCollection    |
| preprocessing/feature provenance                | required    | FitResult state                   |
| model/backend and hyperparameters               | required    | stats linear_regression           |
| performance with uncertainty                    | required    | metrics/uncertainty               |
| diagnostics                                     | required    | DiagnosticResult                  |
| calibration or conformal assumptions where used | conditional | typed calibration/conformal state |
| explanation limitations                         | conditional | causal_interpretation=FALSE       |

```
cat("report written:", basename(report_file), "|", file.size(report_file), "bytes\n")
```

report written: file7f1050e66553.md | 2310 bytes

```
policy <- smf_deprecation_policy()
tab_smf(data.frame(
```

```

item = c("API freeze", "stable release", "policy"),
value = c(policy$api_freeze, policy$stable_release, policy$policy)
), caption = "Current deprecation policy: the API frozen at 1.0.0 does not change without a c

```

Table 4.2: Current deprecation policy: the API frozen at 1.0.0 does not change without a documented lifecycle.

| item           | value                                                                                                                                                    |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| API freeze     | 1.0.0                                                                                                                                                    |
| stable release | 1.0.2                                                                                                                                                    |
| policy         | No public symbol stable at 1.0.0 is removed or changes scientific meaning in 1.x without an explicit lifecycle, migration path, and validation evidence. |

```

cat("manifest available:", !is.null(smf_manifest(fit_p6)), "\n")

```

manifest available: TRUE

## 5 Synthesis: the complete flow in one table

| Step                   | Question                                | Central tool                                                                                   | Evidence                                     |
|------------------------|-----------------------------------------|------------------------------------------------------------------------------------------------|----------------------------------------------|
| 1. Declare             | What are the target, unit and design?   | <code>smf_data_spec</code> ,<br><code>smf_design_spec</code>                                   | contracts refuse misuse                      |
| 2. Audit               | What is discrepant?                     | <code>smf_audit_data</code> +<br>manual diagnostics                                            | missingness, outliers,<br>heteroscedasticity |
| 3. Resample            | What is a “new observation”?            | <code>smf_group_vfold_cv</code> ,<br><code>smf_time_cv</code> ,<br><code>smf_spatial_cv</code> | leakage measured, not<br>presumed            |
| 4. Prepare             | Impute? Centre?<br>Balance?             | <code>smf_fit_preprocessor</code> ,<br><code>smf_apply_imbalance</code>                        | declared and<br>stress-tested decisions      |
| 5. Model               | What is the functional form?            | <code>smf_fit_experiment</code>                                                                | truth recovered,<br>residuals diagnosed      |
| 6. Compare             | Which configuration/model?              | <code>smf_tune</code> ,<br><code>smf_benchmark</code>                                          | same resample,<br>explicit rule              |
| 7. Explain             | What does the model use?                | <code>smf_permutation_importance</code> ,<br><code>smf_pdp</code> , <code>smf_ale</code>       | stability across folds                       |
| 8. Measure uncertainty | How much to trust?                      | conformal, bootstrap,<br>Bayes/GP                                                              | measured coverage,<br>typed semantics        |
| 9. Serve               | How to deploy without losing the model? | bundle, batch, tracker                                                                         | live vs bundle equality                      |
| 10. Report             | What was recorded?                      | <code>smf_report</code> , manifest                                                             | checklist and<br>provenance                  |

## 6 Appendix A — API coverage, computed

The code below reads this tutorial’s own files and measures how many of the 246 exported functions were **called**, per category. It is a measure of the material, not a promise.

```
files <- list.files("en", pattern = "^part-.*\\.qmd$", full.names = TRUE)
code <- paste(unlist(lapply(files, readLines, warn = FALSE)), collapse = "\n")
calls <- unique(unlist(regmatches(code, gregexpr("smf_[a-zA-Z0-9_]+", code))))
api <- smf_api_catalog()
covered <- intersect(calls, api$function_name)
missing_fns <- setdiff(api$function_name, covered)
cov_cat <- as.data.frame(table(api$category[api$function_name %in% covered]))
names(cov_cat) <- c("category", "called")
cov_cat$total <- as.integer(table(api$category))
cov_cat$fraction <- cov_cat$called / cov_cat$total
cov_cat <- cov_cat[order(-cov_cat$called), ]
tab_smf(cov_cat, caption = "Functions called in this tutorial, by API category.")
```

Table 6.1: Functions called in this tutorial, by API category.

|    | category              | called | total | fraction |
|----|-----------------------|--------|-------|----------|
| 1  | core                  | 34     | 88    | 0.386    |
| 6  | modeling              | 26     | 35    | 0.743    |
| 5  | inference-uncertainty | 13     | 24    | 0.542    |
| 9  | resampling-features   | 9      | 23    | 0.391    |
| 3  | deep-learning         | 6      | 23    | 0.261    |
| 7  | operations            | 6      | 17    | 0.353    |
| 8  | reporting-validation  | 5      | 11    | 0.455    |
| 10 | tuning-benchmark      | 5      | 10    | 0.500    |
| 2  | data-design           | 4      | 9     | 0.444    |
| 4  | explainability        | 4      | 6     | 0.667    |

```
cat("called:", length(covered), "of", nrow(api), "exported functions (",
    fmt(100 * length(covered) / nrow(api), 1), "% )\n")
```

called: 112 of 246 exported functions ( 45.5 % )

```
gap_tbl <- data.frame(
  category = api$category[match(missing_fns, api$function_name)],
```

```

    function_name = missing_fns
  )
gap_tbl <- gap_tbl[order(gap_tbl$category, gap_tbl$function_name), ]
tab_smf(gap_tbl, caption = "Exported functions not called in this tutorial, by category (each

```

Table 6.2: Exported functions not called in this tutorial, by category (each has its own vignette or help page).

|    | category | function_name               |
|----|----------|-----------------------------|
| 2  | core     | smf_autoencoder             |
| 4  | core     | smf_backend_capabilities    |
| 14 | core     | smf_blocked_cv              |
| 17 | core     | smf_check_pseudoreplication |
| 18 | core     | smf_chunk_plan              |
| 19 | core     | smf_cnn1d                   |
| 20 | core     | smf_cnn2d                   |
| 27 | core     | smf_dist_cdf                |
| 28 | core     | smf_dist_median             |
| 29 | core     | smf_dist_sample             |
| 30 | core     | smf_dist_sd                 |
| 31 | core     | smf_dist_variance           |
| 50 | core     | smf_end_run                 |
| 53 | core     | smf_explanation_agreement   |
| 54 | core     | smf_explanation_domain      |
| 55 | core     | smf_export_onnx             |
| 59 | core     | smf_from_json               |
| 60 | core     | smf_from_list               |
| 61 | core     | smf_get_run                 |
| 62 | core     | smf_gru                     |
| 64 | core     | smf_hash                    |
| 68 | core     | smf_list_datasets           |
| 69 | core     | smf_load_dataset            |
| 70 | core     | smf_log_artifact            |
| 71 | core     | smf_lstm                    |
| 75 | core     | smf_monte_carlo_cv          |
| 76 | core     | smf_multimodal              |
| 77 | core     | smf_multitask               |
| 80 | core     | smf_param_dbl               |
| 81 | core     | smf_param_fct               |
| 82 | core     | smf_param_lgl               |
| 87 | core     | smf_plot                    |
| 90 | core     | smf_preprocess_provenance   |
| 91 | core     | smf_probabilistic_spec      |
| 94 | core     | smf_register_backend        |
| 96 | core     | smf_require_capability      |
| 98 | core     | smf_rng_info                |
| 99 | core     | smf_rnn                     |

|     | category              | function_name                |
|-----|-----------------------|------------------------------|
| 101 | core                  | smf_run_gold_validation      |
| 102 | core                  | smf_run_reference_validation |
| 103 | core                  | smf_save_manifest            |
| 106 | core                  | smf_select_compromise        |
| 108 | core                  | smf_spec_hash                |
| 110 | core                  | smf_tcn                      |
| 111 | core                  | smf_to_json                  |
| 112 | core                  | smf_to_list                  |
| 113 | core                  | smf_to_yaml                  |
| 116 | core                  | smf_transfer_learning        |
| 117 | core                  | smf_transformer              |
| 122 | core                  | smf_vae                      |
| 125 | core                  | smf_validate_design          |
| 131 | core                  | smf_vit                      |
| 132 | core                  | smf_warning_record           |
| 133 | core                  | smf_with_seed                |
| 23  | data-design           | smf_data_hash                |
| 24  | data-design           | smf_data_iterator            |
| 25  | data-design           | smf_design_summary           |
| 26  | data-design           | smf_detect_leakage           |
| 127 | data-design           | smf_validate_schema          |
| 32  | deep-learning         | smf_dl_architecture          |
| 33  | deep-learning         | smf_dl_checkpoint            |
| 34  | deep-learning         | smf_dl_determinism           |
| 35  | deep-learning         | smf_dl_device                |
| 36  | deep-learning         | smf_dl_distributional_head   |
| 37  | deep-learning         | smf_dl_early_stopping        |
| 38  | deep-learning         | smf_dl_ensemble              |
| 39  | deep-learning         | smf_dl_integrated_gradients  |
| 40  | deep-learning         | smf_dl_load_checkpoint       |
| 41  | deep-learning         | smf_dl_mc_dropout            |
| 42  | deep-learning         | smf_dl_mixed_precision       |
| 43  | deep-learning         | smf_dl_optimizer             |
| 45  | deep-learning         | smf_dl_resume                |
| 46  | deep-learning         | smf_dl_saliency              |
| 47  | deep-learning         | smf_dl_save_checkpoint       |
| 48  | deep-learning         | smf_dl_scheduler             |
| 49  | deep-learning         | smf_dl_spec                  |
| 22  | explainability        | smf_counterfactual           |
| 52  | explainability        | smf_explain                  |
| 5   | inference-uncertainty | smf_bayes_compare            |
| 6   | inference-uncertainty | smf_bayes_diagnostic_flags   |
| 7   | inference-uncertainty | smf_bayes_draws              |
| 8   | inference-uncertainty | smf_bayes_epred              |
| 9   | inference-uncertainty | smf_bayes_loo                |
| 10  | inference-uncertainty | smf_bayes_pp_check           |
| 11  | inference-uncertainty | smf_bayes_prior              |

|     | category              | function_name                   |
|-----|-----------------------|---------------------------------|
| 13  | inference-uncertainty | smf_bayes_stacking              |
| 119 | inference-uncertainty | smf_uncertainty_compare         |
| 120 | inference-uncertainty | smf_uncertainty_report          |
| 121 | inference-uncertainty | smf_uncertainty_spec            |
| 3   | modeling              | smf_available_model_adapters    |
| 12  | modeling              | smf_bayes_prior_predictive      |
| 21  | modeling              | smf_conformal_predict_probably  |
| 44  | modeling              | smf_dl_predict_tensor           |
| 51  | modeling              | smf_evaluate                    |
| 58  | modeling              | smf_fit_model                   |
| 88  | modeling              | smf_predict_distribution        |
| 89  | modeling              | smf_predict_model               |
| 129 | modeling              | smf_vetiver_model               |
| 16  | operations            | smf_bundle_info                 |
| 63  | operations            | smf_hardware_info               |
| 66  | operations            | smf_iterator_next               |
| 67  | operations            | smf_iterator_reset              |
| 73  | operations            | smf_map_iterator                |
| 84  | operations            | smf_persistence_spec            |
| 85  | operations            | smf_pins_fetch_bundle           |
| 86  | operations            | smf_pins_publish_bundle         |
| 115 | operations            | smf_tracking_spec               |
| 130 | operations            | smf_vetiver_pin_write           |
| 134 | operations            | smf_write_plumber               |
| 74  | reporting-validation  | smf_missingness_report          |
| 92  | reporting-validation  | smf_publication_plot            |
| 93  | reporting-validation  | smf_publication_table           |
| 95  | reporting-validation  | smf_reporting_spec              |
| 100 | reporting-validation  | smf_run_crosslang_validation    |
| 124 | reporting-validation  | smf_validate_crosslang_fixtures |
| 1   | resampling-features   | smf_apply_representation        |
| 15  | resampling-features   | smf_build_representation        |
| 56  | resampling-features   | smf_external_split              |
| 57  | resampling-features   | smf_feature_stability           |
| 65  | resampling-features   | smf_import_split_manifest       |
| 72  | resampling-features   | smf_make_resampler              |
| 78  | resampling-features   | smf_nested_resampler            |
| 97  | resampling-features   | smf_resample_manifest           |
| 107 | resampling-features   | smf_select_features             |
| 109 | resampling-features   | smf_split_manifest              |
| 114 | resampling-features   | smf_trace_features              |
| 123 | resampling-features   | smf_validate_bootstrap_design   |
| 126 | resampling-features   | smf_validate_resampling_design  |
| 128 | resampling-features   | smf_validate_split              |
| 79  | tuning-benchmark      | smf_nested_tune                 |
| 83  | tuning-benchmark      | smf_pareto_front                |
| 104 | tuning-benchmark      | smf_search_grid                 |

|     | category         | function_name      |
|-----|------------------|--------------------|
| 105 | tuning-benchmark | smf_search_random  |
| 118 | tuning-benchmark | smf_tune_objective |

```

cov_long <- data.frame(
  category = rep(cov_cat$category, 2),
  type = rep(c("called", "not called"), each = nrow(cov_cat)),
  n = c(cov_cat$called, cov_cat$total - cov_cat$called)
)
ggplot(cov_long, aes(stats::reorder(category, n), n, fill = type)) +
  geom_col(position = "stack") +
  coord_flip() +
  scale_fill_manual(values = c(called = cores_smf[["primaria"]],
                                `not called` = cores_smf[["claro"]])) +
  labs(x = NULL, y = "exported functions", fill = NULL,
       title = "API coverage of this tutorial") +
  theme(legend.position = "top")

```

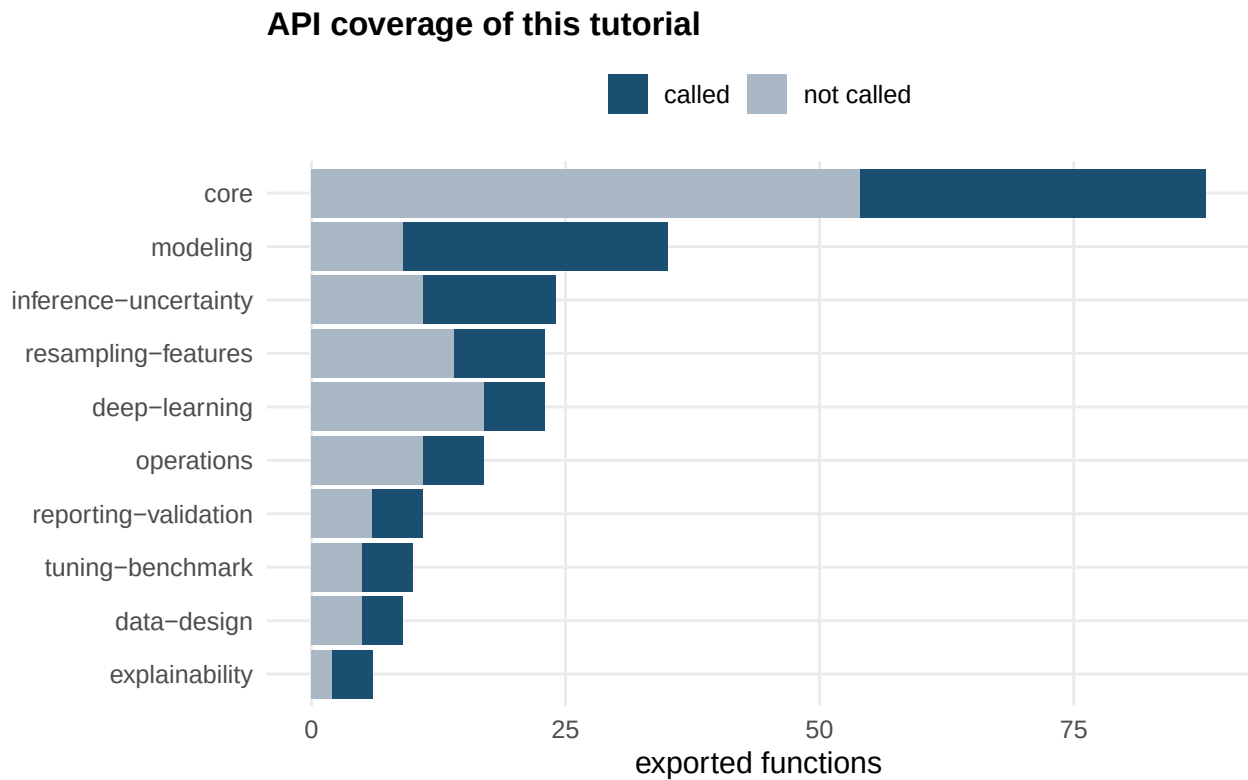


Figure 6.1: API coverage by category: the gaps are declared, not hidden.

## **i** Reading

The uncalled functions belong mostly to three groups: (1) variants for structures that require specific data (DL tensors, `smf_dl_train_tensor`, `smf_dl_gradcam` for images); (2) optional backends that require external packages (`smf_tracker_mlflow`, `smf_conformal_probably`); (3) governance and catalogue utilities (`smf_release_candidate_audit`, `smf_backend_matrix` already used in Part 1). Every gap has a reason — and the reason is declared, as the material's principle demands.

## 7 Appendix B — Minimal glossary

| Term                           | Operational definition                                                                               |
|--------------------------------|------------------------------------------------------------------------------------------------------|
| <b>Contract</b>                | Explicit declaration the package verifies at execution; violating it raises an error, not a warning. |
| <b>Leakage</b>                 | Evaluation information used to build the model; produces optimistic metrics.                         |
| <b>Resampling</b>              | Procedure that defines what counts as a new observation (holdout, folds, groups, time, space).       |
| <b>Fold-safe preprocessing</b> | Statistics (median, scale) estimated only on each fold's training data.                              |
| <b>Imbalance</b>               | Classes with very different frequencies; accuracy stops being a useful metric.                       |
| <b>Calibration</b>             | Agreement between declared probability and observed frequency (ECE measures the gap).                |
| <b>Coverage</b>                | Frequency with which the interval contains the true value; the conformal guarantee is marginal.      |
| <b>Aleatoric uncertainty</b>   | Irreducible variability of the phenomenon.                                                           |
| <b>Epistemic uncertainty</b>   | Model ignorance, reducible with more data.                                                           |
| <b>Bundle</b>                  | Self-contained artefact (model + preprocessing + metadata) for inference outside the session.        |
| <b>Manifest</b>                | Provenance record: hashes, versions, seeds and decisions of a run.                                   |

## 8 Appendix C — Execution environment

```
cat("sciModelFlowR:", as.character(packageVersion("sciModelFlowR")), "\n")
```

sciModelFlowR: 1.0.2

```
cat("R:", R.version.string, "\n")
```

R: R version 4.6.0 (2026-04-24 ucrt)

```
cat("platform:", R.version$platform, "\n")
```

platform: x86\_64-w64-mingw32

```
cat("core packages:", paste(c("ggplot2", "knitr", "patchwork"), collapse = ", "), "\n")
```

core packages: ggplot2, knitr, patchwork

```
cat("executed at:", format(Sys.time(), "%Y-%m-%d %H:%M"), "\n")
```

executed at: 2026-09-19 00:53

## 9 Part 6 exercises

### Exercise 6.1 — Prior strength

Refit the conjugate model with `beta_sd = c(0.5)` (tight prior) and with `beta_sd = c(20)` (wide prior). Compare the posteriors with the truth (5.2). With `n = 30`, does the prior still matter?

### Exercise 6.2 — GP with fixed hyperparameters

Run the GP with `optimize = FALSE` and `length_scale = 0.05`. Does RMSE get worse? What does this teach about the spatial range scale (truth: 0.25)?

### Exercise 6.3 — BART versus MLP

Compare BART and MLP on the same `dl_tiny` split (RMSE and time). In which situation would you prefer each one?

### Exercise 6.4 — Opaque bundle

Save the same model in `opaque` mode, try to load it without `trusted = TRUE` and read the error. Then load it with explicit trust and check prediction equality. Why must trust be explicit?

### Solutions

**6.1.** With a tight prior (0.5), the posterior is pulled toward 5.0; with a wide prior (20), it stays close to the likelihood (~5.3). With `n = 30`, the prior still moves the result — the size of the effect depends on the ratio between prior information and data.

**6.2.** With range 0.05 (smaller than the true 0.25), the GP sees no correlation between neighbours and falls back to the mean level; RMSE rises. The range is a **scientific hyperparameter**, not a technical detail.

**6.3.** BART and MLP reach errors close to the noise (0.15); the MLP needs epochs and a seed, BART needs an MCMC chain. Prefer BART for small tabular data and MLP for structured inputs (image, sequence) or large datasets.

**6.4.** The opaque bundle loads native objects that can execute code; `trusted = TRUE` is the explicit signature of trust. The error without the argument is the security boundary working.